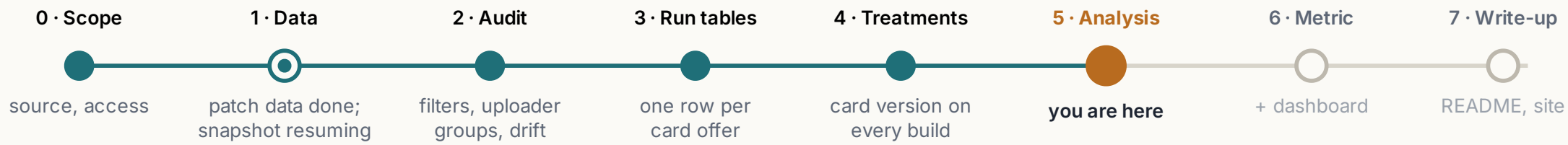
What did the balance patches actually do?

The plan for the first analysis: the dataset, what we're trying to estimate, and the reasoning behind each modelling choice.

Slay the Spire 2 early access · community run data (Spire Codex export) · September 2026

THE FOREST

Four phases of plumbing are done. Now the analysis starts.



WHAT THE PLUMBING BOUGHT US

- Every reward offer, picked or not, tagged with the card version live on that run's build.
- A clean before/after window for each of the 50 numeric card tweaks.
- A list of what to leave out of each comparison, and known data problems written down.

PHASE 5, FIRST TWO STEPS

- **Offer-rate check**: is the "experiment" intact?
- **Pick-rate difference-in-differences** for the numeric tweaks, then effect vs. change size. This is the minimum result worth showing.

THE QUESTION

When a card changed, how did players respond, and did it matter?

Community stats sites report "this card's win rate." That number mixes up *the card is strong* with *strong runs take this card*.

- Runs that are going well reach more floors, so they see more card rewards.
- Runs in trouble pick differently from runs that are cruising.
- Some cards mostly show up at certain points in a run.

Separating those is the project. Patches help, because each one is an intervention with a known date and a known size.

FOUR QUESTIONS, IN ORDER OF DIFFICULTY

1. **Behavior**: did the pick rate change?
2. **Dose-response**: do bigger changes move picks more?
3. **Strength**: did picking the card help runs win more or less?
4. **Perception vs. strength**: do players over- or under-react?

Phase 5 starts with 1 and 2. The data measures them best: pick decisions are frequent, and a single card barely moves a run's outcome.

THE DATASET

Raw run files uploaded by players, one row per card offered

947,885
run records on the first 949 export pages (submitted up to ~19 July 2026)

727,714
distinct runs once co-op copies are merged

~75%
of distinct runs are in the primary population (next panel)

28
game builds, two branches (beta and stable)

WHAT A RUN FILE GIVES US

- Every card offered on every reward screen, and whether it was picked. This is the key feature: we see the cards players *turned down*.
- Build, character, ascension, act, upgraded or not, win or loss.
- Nothing about what happened in combat (damage per card, plays per turn).

PRIMARY POPULATION

- Solo, no modifiers, no mods; a released build; not started before that build came out; begins at Neow.
- Offers from fight rewards: normal, elite, boss, `?` fights, and event-then-fight floors (flagged).
- Abandoned runs count for pick rates, not for win rates.

Table: `offers_with_version` in `data/runs/runs.duckdb`. The snapshot download is still running; the numbers get rebuilt once it finishes.

THE TREATMENTS

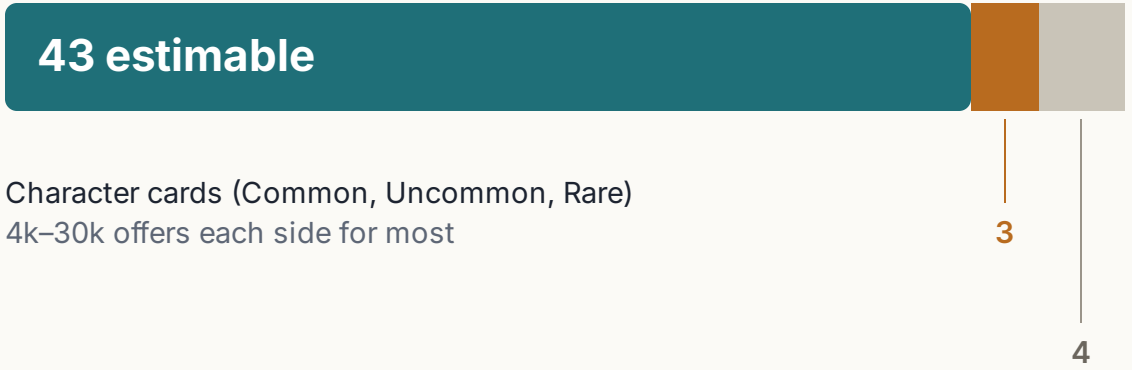
50 small number changes, each with a clean before and after

The patch history turns into 262 card change events (plus 64 bug fixes that changed how a card plays). For the first result we use only **numeric tweaks**: same card, one number moved.

- **Why tweaks first:** the size of the change is known (Bulwark block 13 → 12), so we can ask whether bigger changes do more. Reworks change what a card *is*, so they get separate before/after estimates.
- **Clean windows:** compare the stable build before the patch with the stable build after it, where the card differs only by its own change.

STABLE WINDOW	TWEAKS
v0.99.1 → v0.103.2 (mid-April)	31
v0.103.3 → v0.107.1 (mid-June)	19

HOW MANY WE CAN ESTIMATE



Character cards (Common, Uncommon, Rare)
4k–30k offers each side for most

3 colorless, too few offers
Eternal Armor, Seeker Strike, Fasten

4 never in solo rewards
Believe in You (co-op only),
Falling Star (starter),
Minion Strike/Sacrifice (tokens)

WHY THIS CAN WORK

Within a build, the game's dice pick the offers. Across builds, nothing is random.

WHAT THE RNG GIVES US

Which three cards appear on a reward screen is decided by the random number generator, not by the player. So on one build, "this run was offered Pounce" is close to a coin flip, and the run file records the turned-down offers too.

It's like an A/B test where the platform decides who sees what. The player still decides whether to take it (the "compliance" part).

WHAT CHANGES BETWEEN BUILDS, BESIDES THE CARD

- Other cards and enemies changed in the same patch.
- Players get better, and they copy each other (the meta drifts).
- The mix of uploaders shifts (the SlayTheStats mod group).
- Rates drift within a single build over calendar time.

> A plain before/after would blame the card for all of the right-hand column. The rest of the design is about removing it.

STEP 1 · OFFER-RATE CHECK

First make sure the change didn't change who gets offered the card

A number tweak should change how often a card is *picked*, not how often it's *offered*. If the offer rate moved, something else changed too (rarity, the card pool, a reward rule), and the pick comparison for that card is suspect.

HOW

- For each card and build block: its share of that character's reward offers.
- Ratio after/before for the changed card, set against the same ratio for every unchanged card of that character.
- Flag a card when its ratio sits far outside the unchanged cards' spread *and* the difference is well beyond counting noise.

WHY IT MATTERS

The whole design leans on the offer being set by the dice. This checks that assumption card by card, before trusting any pick-rate result.

INTERVIEW VERSION

It's the sample-ratio-mismatch check from A/B testing: before reading the metric, confirm that assignment worked as designed.

STEP 2 · DIFFERENCE-IN-DIFFERENCES

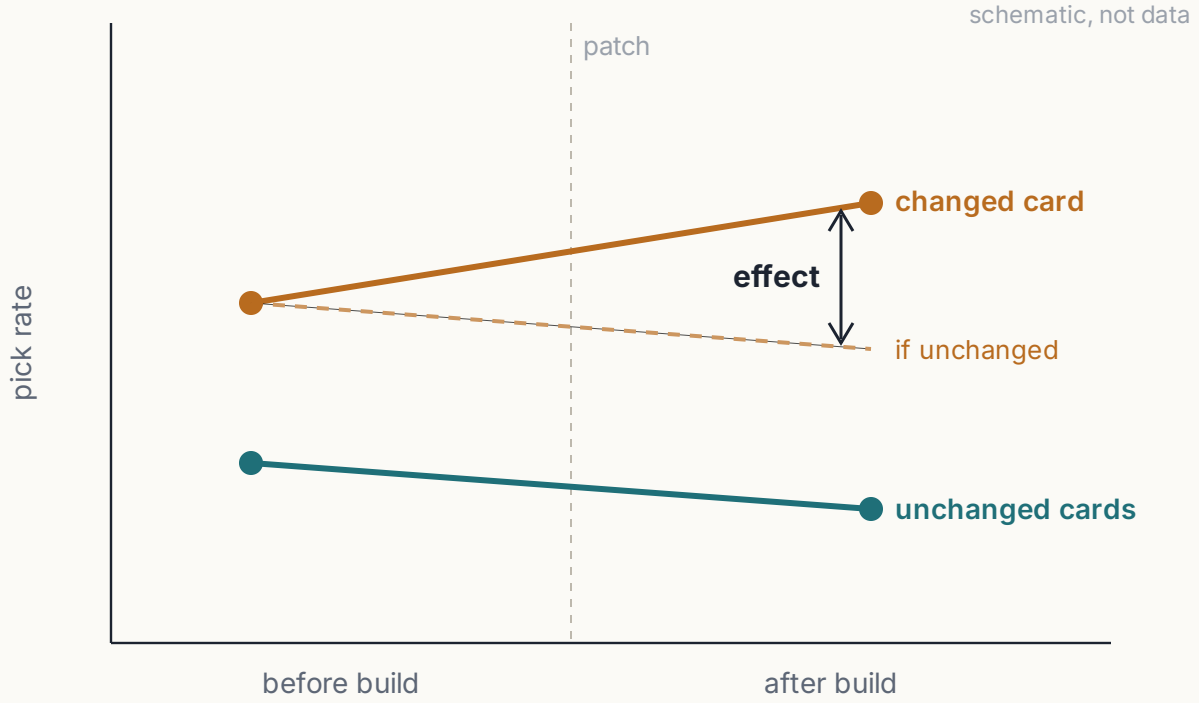
Compare the changed card's shift with the shift in cards that didn't change

Unchanged cards of the same character live through the same patch, the same players, the same meta. Their before/after shift estimates what would have happened to the changed card anyway. What's left over is attributed to the change.

$$\text{effect} = (\text{changed card: after} - \text{before}) - (\text{unchanged cards: after} - \text{before})$$

THE ONE ASSUMPTION: PARALLEL TRENDS

Without the change, the card's pick rate would have moved the way the unchanged cards' did. It can't be proven, but it can be checked before the change (step 2b).



STEP 2 · THE MODEL, TERM BY TERM

One regression per changed card, on every offer in its window

$$\text{picked} = \text{card baseline} + \text{build-block shift} + \text{act, ascension} + \beta \times (\text{changed card} \times \text{after}) + \text{noise}$$

TERM	WHAT IT SOAKS UP	WHY IT'S THERE
Card × upgraded baseline	Each card's usual pick rate, separately for upgraded offers	Cards differ hugely in pick rate, and an upgraded card is almost a different card. We only want changes, not levels.
Card × uploader group	SlayTheStats-mod uploaders ranking cards differently	The groups' typical gap (5–7 points) is as big as many patch effects.
Build block × uploader group	Everything that moved all cards at once, in each group	This is the "unchanged cards" half of the difference.
Act, ascension	Where in the run, and how hard the run is	Offers are random, so these mostly reduce noise. They also guard against a shift in the ascension mix.
β	The effect: change in the card's pick probability, in percentage points, relative to unchanged cards.	

MODELLING CHOICE

Why a straight-line model for a yes/no outcome

"Picked" is 0 or 1, so logistic regression is the reflex. For this job a linear probability model (ordinary least squares on 0/1) is the better tool:

- **Readable.** β is in percentage points: "the buff raised the pick rate by 3 points."
- **The DiD term means what it says.** In a logit the interaction coefficient is not the difference-in-differences in probability. The translation depends on the baseline rate.
- **Fixed effects are cheap.** Hundreds of card and block dummies over millions of offers are routine for OLS with absorbed fixed effects.
- **Its usual weakness doesn't bite.** Predictions outside 0–1 don't matter when we only use differences near observed rates.

WHERE THE SCALE DOES MATTER

A card picked 5% of the time can't drop by 10 points. When comparing effects *across* cards (dose-response), put them on a log-odds scale, where a card near 5% and one near 50% can move comparably. Report both.

A CATCH WORTH SAYING OUT LOUD

"Parallel trends" depends on the scale. Parallel in percentage points isn't parallel in log-odds. That's another reason to show both and check that the conclusions agree.

MODELLING CHOICE

Who counts as a control, and how the controls can go wrong

THE CONTROL GROUP

- **Same character**: a card's competitors are the other cards of that character, offered on the same screens to the same players.
- **Unchanged in the window**: drop every card of that color that changed between the two builds (`control_exclusions`).
- **Bug-fix flags**: cards whose behavior changed through a bug fix stay in, and a sensitivity run drops them.

Upgrade-only tweaks (Serpent Form, Royalties, Untouchable v0.107.1, most of Death March) are estimated on upgraded offers. Their un-upgraded offers become a free placebo.

TWO WAYS CONTROLS CAN BE CONTAMINATED

Competition on the screen. Three cards share a screen and you take at most one. If Pounce gets better, the cards next to it get picked a bit less, which makes Pounce's effect look slightly bigger.

Size: only a few percent of control offers share a screen with a given card. Check: rerun without them.

Synergy. An unchanged card whose partner was buffed isn't really unchanged. We haven't listed synergies, so check by dropping one control card at a time and watching whether β moves.

MODELLING CHOICE

Two kinds of uploaders, who value cards differently

Runs uploaded by the SlayTheStats mod (`_sts` tag) rank cards differently from the rest, already before the mod showed community stats. That points to *who* these players are (selection), not to the mod's tooltips.

BUILD	SD OF PER-CARD Z	SPEARMAN	TYPICAL GAP
v0.103.2 (before tooltips)	3.76	0.93	4.6 pts
v0.107.1 (after)	4.95	0.86	7.4 pts

And the tagged share moves with upload timing. If tagged runs are more common after a patch than before, a plain before/after picks up a fake "effect" just from the mix changing.

WHAT THE MODEL DOES ABOUT IT

Each group gets its own card baselines and its own before/after shift. β is then the change common to both, and it can't come from the mix moving.

CHECKS

Estimate within each group separately. They should agree in sign and roughly in size. Also run a pooled version with no group terms, to show how much the adjustment matters.

MODELLING CHOICE · UPDATED WITH FIRST-PASS RESULTS

Error bars: cluster by run, and know what the ratio measures

A run gives dozens of offers from the same player and deck, so errors are clustered by run. On the first pass (43 tweaks), clustered SE ÷ plain SE ranged **0.59–1.26**, median 0.99. But that ratio mixes two things:

- **Unequal noise per card.** Plain OLS assumes every offer is equally noisy. A 0/1 outcome's noise is p(1–p), so a card picked 5% of the time is quieter than one picked 50%. Clustered errors correct for that too. It explains almost all of the spread: the ratio tracks √(p(1–p)) with correlation 0.96.
- **Correlation within a run.** Measured against robust errors, which already allow for that, clustering widens the bars by about **3%** (median 1.03, range 1.00–1.20). The most affected are common cards offered many times per run (Crescent Spear 1.20, Uproar 1.19, Collision Course 1.13).
- A taste shared by the whole run cancels between the changed card and its controls (synthetic test). What's left is a run's taste for one particular card.

> **INTERVIEW VERSION**
>
> Cluster where the treatment's errors are correlated, and compare against robust errors rather than plain ones, or you'll misread unequal noise as correlation. Here clustering cost about 3% in precision.

THE NEXT LEVEL UP: UPLOADERS

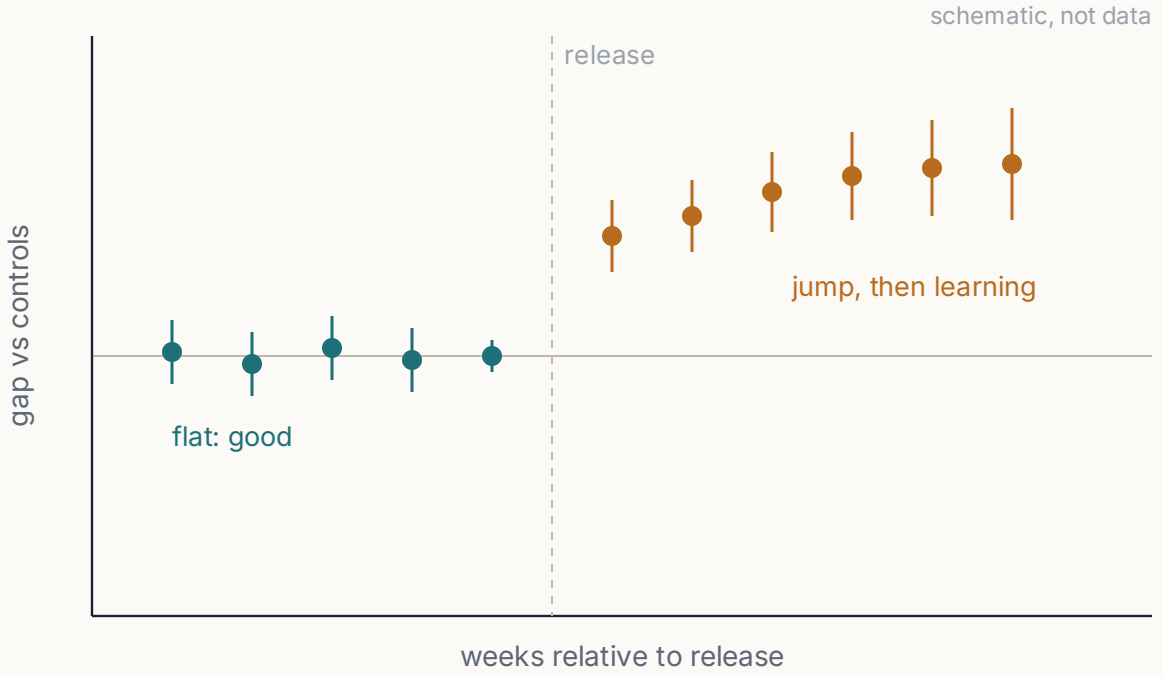
Runs by the same person aren't independent either. The current tables have no uploader IDs (their pages predate `player_token`), so the heavy-uploader check waits for the rebuild on the full snapshot.

Duplicate uploads: 106 of 577,092 solo runs (0.02%); 18 of those are different runs sharing a `run_key`.

Plot the gap week by week, before and after the release

Split each window by week (relative to the new build's release) and estimate the changed card's gap to the controls in each week.

- **Before release:** the gap should be flat. A slope means the card was already drifting, and the DiD would credit that drift to the patch.
- **After release:** the effect is a path, not a number. There's a jump when players read the notes, then learning (Corrosive Wave kept sliding for weeks).
- Old builds keep being played after the release, so "before build, after date" weeks exist. They help separate the build from the calendar.



STEP 3 · DOSE-RESPONSE

Put all the tweaks on one plot: effect against size of change

- **x**: relative change in the card's main number, as log(new / old). So 6 → 8 and 8 → 6 are the same size in opposite directions.
- **y**: the card's β on the log-odds scale, so cards with low and high baseline pick rates are comparable.
- **Fit**: a weighted line through the ~43 points (weights $1/SE^2$), with separate slopes for buffs and nerfs. Expect real scatter around the line, since +1 damage is worth more on some cards than others.
- **The intercept is interesting**: a nonzero value at "no change" would mean being in the patch notes moves picks by itself (attention).

KEPT OFF THE LINE, SHOWN SEPARATELY

- **Mixed-direction tweaks**: Leading Strike (less damage, more Shivs), Reflect (lower base block, bigger upgrade bonus).
- **Upgrade-only tweaks**: plotted on their upgraded offers. Un-upgraded offers should show ~0, and anything else is the attention effect again.
- Cost changes and reworks: not on this axis.

WHY THIS IS THE HEADLINE

One card's estimate is noisy. Forty-odd estimates lining up with change size is hard to explain as noise, and it answers "how much pick rate does +1 buy?"

WHAT COULD GO WRONG

Each threat gets a specific check

THREAT	CHECK
The patch changed how often the card is offered	Offer-rate check, before any pick result is read
The card was already trending	Weekly gaps before release (pre-trends)
Uploader mix shifted	Group-specific baselines and shifts; estimates within each group
A few heavy uploaders dominate	Drop the top uploaders by `player_token`; clustered errors
Same run uploaded twice	Keep one copy per `run_key`
Changed card steals picks from its screen-mates	Rerun without control offers that shared a screen with it
A control's synergy partner changed	Leave-one-control-out
Bug fixes changed a control's behavior	Sensitivity run excluding flagged cards
Scale choice (points vs log-odds)	Report both; conclusions should agree
43 estimates: some look "significant" by chance	Lead with the dose-response pattern; partial pooling later

AFTER THE MINIMUM RESULT

From "what did players do" to "what did the card do"

- **Win effect of being offered** (intent-to-treat): runs offered the card vs. runs not, within a build. Randomized by the dice.
- **Win effect of picking** (instrumental variable): the offer is the instrument, so the estimate is free of "good runs pick it." Then compare it before vs. after the patch.
- **Perception vs. strength**: set pick-rate effects against strength effects. Do players overreact to buffs?
- **Partial pooling** (hierarchical Bayes): rarely offered cards borrow strength from similar cards instead of producing extreme estimates.
- **Staggered rollout**: beta got changes weeks before stable, a second natural experiment. **Reworks**: before/after, reported separately.

WHY WINS COME SECOND

One card is a small part of a 50-floor run, so its effect on winning is small next to the run-to-run noise.

The instrumental estimate divides the offer's effect by how much the offer changes picking. When that number is small, the uncertainty blows up. Pick rates, by contrast, are measured directly at every screen.

THE 30-SECOND VERSION

How I'd say it in an interview

> "Slay the Spire 2 rebalanced cards every couple of weeks during early access, and players upload full run files that record every card they were offered, including the ones they turned down. Offers are random within a build, so I treat each card change as an experiment. I compare the changed card's pick rate before and after the patch against unchanged cards of the same character. That's a difference-in-differences, with errors clustered by run and separate baselines for two uploader populations that value cards differently. I check that offers didn't move, that there were no trends before the patch, and I put the 40-odd effects on one dose-response curve: how much pick rate one point of damage buys."

BEFORE RUNNING FOR REAL

Open items

PIPELINE (CCC)

- Add a regression library to the `analysis` extra (pyfixest: fixed effects plus clustered errors at millions of rows).
- Build `offers_with_version` on deduplicated runs (one per `run_key`). Today it can count a twice-uploaded solo run twice.
- Rebuild the run tables once the snapshot finishes.

DECISIONS (PHIL)

- Confirm the "main number" and buff/nerf direction for each tweak (the notebook prints a table to review).
- Ascension as a control: bins or every level.
- How many heavy uploaders to drop in the robustness check.
- Write the chosen design into `SPIROMETRY.md` once the first run looks sane.

Notebook: `notebooks/02_pick_rate_did.ipynb`. The code is written to run on the current 949-page tables.